3장 깃 개념 잡기

이 장에서는 깃의 저장소 개념을 알아보고, 직접 생성해 봅니다. 깃을 효과적으로 잘 사용하려면 저장소의 동작 원리를 이해하는 것이 중요합니다.

3.1 깃 저장소 생성

3.2 워킹 디렉터리

3.3 스테이지

3.4 파일의 상태 확인

3.5 파일 관리 목록에서 제외: .gitignore

3.6 깃 저장소 복제

3.7 정리

3.1 깃 저장소 생성

깃은 작성된 소스 코드 파일의 모든 변경 사항을 관리합니다. 그렇다면 깃은 파일의 변경 내역을 어떻게 저장하고 관리할 수 있을까요? 깃은 이러한 변경 사항을 전용 저장소(repository)(리포지터리)에 저장합니다. 이 저장소는 일반적으로 사용하는 폴더와 유사하지만, 조금 차이가 있습니다. 깃의 동작 방식을 이해하려면 저장소 동작 원리를 확실히 알아야 합니다. 이제부터 하나씩 알아봅시다.

3.1.1 폴더와 깃 저장소

컴퓨터의 파일과 폴더는 운영 체제의 파일 시스템에 의존하여 동작합니다. 파일 시스템은 하드디스크 같은 장치에 데이터를 저장하고 관리합니다. 그중 폴더는 파일 여러 개를 하나로 관리할 수 있는 논리적 개념입니다. 마치 파일을 그룹으로 묶어 놓은 것과 같습니다.

그렇다면 깃은 파일의 변경 내역을 어떻게 저장하고 관리할 수 있을까요? 깃 저장소는 외형적으로 폴더와 유사합니다. 사용자 입장에서는 일반 폴더와 깃 저장소를 구별 없이 모두 동일하게 사용할 수 있습니다. 하지만 깃 저장소는 내부적으로 구조가 다릅니다. 깃 저장소에는 별도의 숨겨진 영역(숨겨진 폴더)이 있는데, 여기에 버전 관리 시스템(VCS, Version Control System)에 필요한 파일 변경 이력을 기록합니다. 저장소는 프로젝트의 모든 리비전(revision)(개정)과 히스토리를 가진 데이터베이스와 같습니다.

즉, 일반적인 폴더와 깃 저장소 차이점은 숨겨진 영역이 있는지 여부입니다.

3.1.2 초기화

그럼 깃 저장소의 숨겨진 영역을 살펴보면서 저장소를 좀 더 알아봅시다. 저장소를 생성하려면 먼저 초기화 작업이 필요합니다. 깃에서 초기화란 이미 존재하는 폴더에 초기화 명령어로 VCS 관리를 위한 숨겨진 영역을 생성하는 작업을 의미합니다.

깃 초기화를 해 봅시다. 먼저 터미널(깃 배시)을 실행합니다.

Note

터미널은 텍스트로 명령어를 입력할 수 있는 대화창입니다. 깃 배시 터미널 프로그램 외에도 윈도에 기본 내장된 CMD, powerShell 등을 사용해도 됩니다. 책에서는 깃 배시를 기준으로 합니다. 윈도 메뉴 말고 바탕화면의 깃 배시 아이콘으로 실행해도 됩니다.

▼ 그림 3-1 깃 배시 아이콘



실습에 필요한 새 폴더를 하나 만들겠습니다. 또는 기존에 있던 폴더에서 시작해도 괜찮습니다. 먼저 실습 폴더를 만들고 해당 폴더로 이동합니다. 책에서는 jinygit03 폴더를 만들고 해당 폴더로 이동했습니다.

```
$ mkdir jinygit03 새 폴더 만들기


$ cd jinygit03 만든 폴더로 이동
```

3.1.3 숨겨진 폴더 = .git 폴더

git init 명령어를 사용하여 일반적인 폴더를 깃이 관리할 수 있는 저장소로 변경했습니다. 깃 저장소를 초기화한다는 것은 별도의 숨겨진 폴더를 하나 추가하고 환경 설정 파일을 생성하는 것입니다. 생성된 숨겨진 폴더를 확인해 봅시다.

먼저 깃 배시 터미널이 실행된 상태에서 ls 명령어로 파일 목록을 출력합시다.

```
$ ls
```

ls 명령어는 파일 목록을 출력하는 리눅스 명령어입니다. 초기화된 폴더에서 ls 명령어를 입력하면 아무 내용도 출력되지 않습니다.

일반적인 ls 명령어는 숨겨진 폴더까지 확인할 수 없습니다. 이번에는 -a 옵션을 추가하여 입력합니다. -a 옵션은 폴더 안의 숨겨진 파일을 같이 출력하라는 의미입니다.

```
$ ls -a
```

```
./  ../  .git/
```

이전 결과와 달리 목록이 화면에 출력되었습니다. 목록을 보니 .git이라는 숨겨진 폴더가 하나 있습니다. 보통 폴더 이름 앞에 점(.)이 있으면 숨겨진 폴더를 의미합니다. 숨겨진 폴더인 .git 폴더에는 깃 저장소에 필요한 모든 뼈대 파일이 담겨 있습니다. 이러한 뼈대 파일들은 깃 초기화를 통하여 자동 생성됩니다.

3.1.4 소스트리와 연결

로컬 컴퓨터에 생성된 깃 저장소를 소스트리와 연동하여 이력을 쉽게 관리할 수 있습니다. 저장소를 소스트리와 연결해 보겠습니다. 먼저 소스트리를 실행합니다.

새 저장소를 생성하여 연결

소스트리에 새 저장소를 생성하여 로컬 저장소와 연결해 보겠습니다. 위쪽 + Create 버튼을 클릭하여 새 저장소를 생성합니다. 새 저장소 생성은 폴더 생성 및 초기화 명령어인 git init를 동시에 실행하는 것과 같습니다.

그럼 새 저장소를 생성해 봅시다. 소스트리 첫 화면에서 + Create 버튼을 클릭합니다.

▼ 그림 3-5 새 저장소 생성



3.2 워킹 디렉터리

깃의 동작을 이해하려면 먼저 워킹 디렉터리(working directory) 개념을 알아야 합니다. 워킹 디렉터리는 다른 용어로 워킹 트리(working tree)라고도 합니다.

3.2.1 워킹 디렉터리란?

깃은 VCS의 특성 때문에 저장 공간을 논리적으로 분리합니다. 깃을 처음 학습할 때는 이러한 논리적인 공간 분리 때문에 어렵게 느끼곤 합니다. 깃은 저장 공간을 크게 '작업을 하는 공간(working)'과 '임시로 저장하는 공간(stage)', '실제로 저장하여 기록하는 공간(repository)'으로 나눕니다. 이렇게 논리적으로 공간을 분리하는 것은 깃의 동작과 이력을 좀 더 효율적으로 처리하기 위해서입니다.

워킹 디렉터리는 '작업을 하는 공간'을 의미합니다. 말 그대로 로컬 저장소에 접근할 수 있으며, 실제로 파일을 생성하고 수정하는 공간입니다. 단순하게 파일을 저장하는 공간이라고 생각하면 됩니다. 워킹 디렉터리는 3.3절에서 설명하는 스테이지 영역과 연결되어 있습니다.

3.2.2 파일의 untracked 상태와 tracked 상태

깃의 추적(tracked) 개념을 알아봅시다. 깃이 다른 VCS보다 뛰어난 것은 지정된 파일들의 모든 것을 추적하는 관리 시스템이기 때문입니다. 깃은 워킹 디렉터리에 있는 파일들을 '추적됨'과 '추적되지 않음' 상태로 구분합니다.

untracked 상태

실제 작업 중인 파일은 워킹 디렉터리 안에 있습니다. 워킹 디렉터리는 현재 작업 중인 소스 코드를 담고 있으며, 운영 체제도 워킹 디렉터리 안에 있는 파일들만 접근하고 수정할 수 있습니다.

워킹 디렉터리는 사용자 작업 공간이라고 생각하면 됩니다. 이 공간에 파일을 추가하거나 수정했다고 해서 깃이 자동으로 관리해 주지는 않습니다. 워킹 디렉터리에 새로 생성된 파일은 모두 추적되지 않음(untracked) 상태입니다. 이 파일을 관리하려면 깃에 추적하라고 통지해 주어야 합니다. 통지하지 않은 파일은 깃에서 따로 추적하지 않습니다.

3.3 스테이지

깃은 여러 단계의 논리적인 저장 공간을 가지고 있습니다. 스테이지(stage)는 '임시로 저장하는 공간'을 의미합니다. 스테이지 영역은 워킹 디렉터리에서 제출된 tracked 파일들을 관리합니다. 이 영역은 나중에 배울 커밋 작업과 연관이 매우 많습니다.

3.3.1 스테이지 = 임시 영역

스테이지는 워킹 디렉터리와 '실제로 저장하여 기록하는 공간' 사이에 있는 임시 영역입니다. 깃은 워킹 디렉터리에서 작업이 끝난 파일을 스테이지로 잠시 복사합니다. 스테이지가 임시 영역이라고 해서 파일의 콘텐츠 내용을 직접 가지고 있지는 않습니다. 단지 커밋하려는 파일의 추적 상태 정보들만 기록합니다.

▼ 그림 3-16 스테이지는 커밋하려는 파일의 추적 정보만 기록



이처럼 임시 영역인 스테이지를 별도로 운영하는 것은 커밋을 빠르게 처리하기 위해서입니다. '실제로 저장하여 기록하는 공간'인 저장소는 스테이지 영역에서 가리키는 파일 내용을 기반으로 변경된 차이점만 기록합니다.

파일들의 스테이지 상태는 status 명령어로 확인 가능합니다. 또는 깃의 git ls-files 같은 명령어로도 확인 가능합니다.

예

```
$ git status
```

```
$ git ls-files --stage
```

스테이지 영역에 등록된 파일들은 또 다시 stage 상태와 unstage 상태로 구분됩니다. 버전 관리에서 제외하고 싶은 파일이 있다면 .gitignore 파일에 등록합니다. .gitignore 파일은 3.5절에서 설명합니다.

3.3.2 파일의 stage 상태와 unstage 상태

워킹 디렉터리에 있는 tracked 상태의 파일들은 스테이지 영역과 긴밀한 상관관계를 맺습니다. 스테이지 영역으로 등록된 모든 파일은 untracked 상태에서 tracked 상태로 변경됩니다. 스테이지는 워킹 디렉터리 안에 있는 파일들의 추적 상태를 관리하는 역할을 수행합니다.

스테이지 영역은 파일을 stage 상태와 unstage 상태로 구분합니다. 깃이 변화 이력을 기록하려면 파일들의 최종 상태가 stage 상태여야 합니다. unstage 상태라면 파일에 변화가 있다는 것을 의미합니다. 즉, 스테이지 영역에 있는 파일과 워킹 디렉터리 안에 있는 파일 내용에 차이가 있을 때는 unstage 상태가 됩니다.

또 넓게 보면 아직 스테이지 영역으로 등록하지 않은 워킹 디렉터리 안의 파일도 unstage 상태라고 생각할 수 있습니다. 이때는 unstage 상태이자 동시에 untracked 상태입니다.

unstage 상태라고 해서 실제 파일이 없어지는 것은 아닙니다. 단지 파일이 수정되어 임시적으로 스테이지 목록에서 제외된 것입니다. git add 명령어를 사용하면 스테이지에 다시 추가할 수 있습니다.

3.3.3 파일의 modified 상태와 unmodified 상태

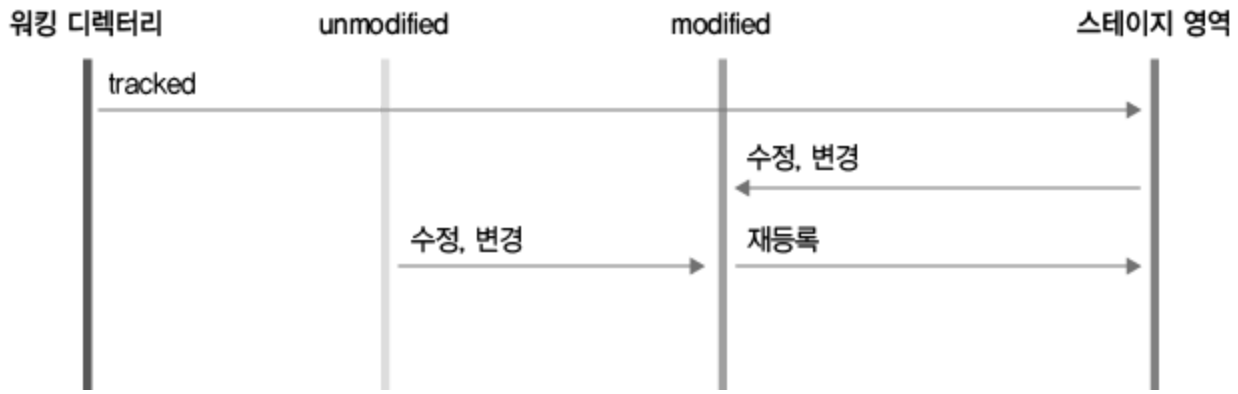
코드를 변경한다는 것은 워킹 디렉터리에서 파일을 수정하는 것을 의미합니다. 파일이 수정되면 워킹 디렉터리와 스테이지 간 내용이 일치하지 않습니다. 따라서 스테이지는 수정한 파일과 원본 파일을 구분하려고 수정함(modified) 상태와 수정하지 않음(unmodified) 상태로 표현합니다. 파일 수정 작업은 스테이지 영역과 긴밀한 상관관계를 맺습니다.

modified 상태

스테이지에 등록된 파일은 깃이 추적 관리합니다. 깃이 실제로 기록한 파일이며, 사실상 버전을 의미합니다. 파일 수정은 개발 과정에서 뗄 수 없는 작업입니다. 개발 작업에서 수많은 코드가 변경되고, 깃을 사용하면 이 변경 내역은 영구적으로 기록됩니다.

앞에서 설명했듯이 깃은 tracked 상태인 파일만 수정 여부를 관리할 수 있습니다. tracked 상태인 파일이 수정되면 스테이지는 파일 상태를 modified 상태로 변경합니다.

▼ 그림 3-17 파일 수정 여부 확인



그러면 수정된 파일은 스테이지에서 잠시 제외됩니다. 깃은 수정 여부만 체크해 주기 때문에 modified 상태로 변경된 파일은 스테이지로 재등록해야 합니다. 수정된 파일을 스테이지 영역으로 다시 적용하려면 git add 명령어로 재등록합니다.

3.4 파일의 상태 확인

상태 개념은 깃의 분리된 저장 영역인 워킹 디렉터리와 스테이지, 추적 여부를 의미합니다. 깃이 이렇게 다양한 저장 영역을 구분해서 가지고 있는 것은 파일들의 상태를 효율적으로 모니터링하기 위해서입니다. 이번에는 파일들의 상태를 모니터링할 수 있는 status 명령어를 알아봅니다.

3.4.1 status 명령어로 깃 상태 확인

파일을 생성하고 수정한다는 것은 변화를 의미합니다. 또 이러한 변화들은 순서가 있습니다. 깃은 각 저장 영역에서 일어나는 다양한 변화를 감시합니다. 그리고 이러한 변화를 감지하고 상태 메시지를 출력합니다.

status 명령어를 사용하면 깃의 상태 메시지를 확인할 수 있습니다. 특히 status 명령어는 많이 사용하는 깃 명령어 중 하나입니다.

터미널(깃 배시)에서 status 명령어를 입력합니다.

```
$ git status 상태 확인


On branch main


No commits yet 커밋이 없다는 메시지

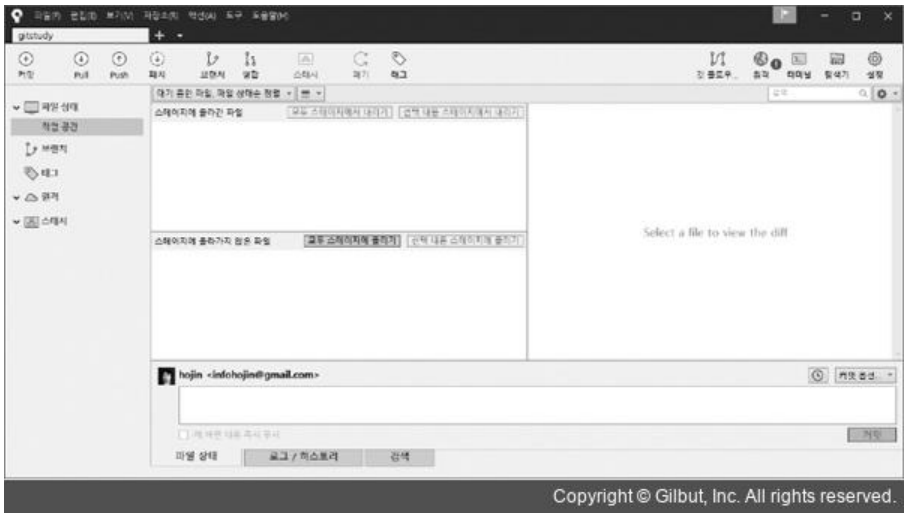

nothing to commit (create/copy files and use "git add" to track) 변경된 내용이 없다는 메시지
```

우리는 처음 깃 저장소를 생성하고, 실습 이후에 아무 작업도 하지 않았습니다. status 명령어를 실행하면 이처럼 변경된 내용과 커밋이 없다고 메시지를 출력할 것입니다. 나중에 커밋 명령을 수행하면 status 명령어로 파일들의 변경된 상태를 확인할 수 있습니다.

3.4.2 소스트리에서 깃 상태 확인

소스트리를 이용하면 콘솔에서 status 명령어를 입력하지 않고도 바로 깃 상태를 확인할 수 있습니다. 소스트리를 실행한 후 왼쪽의 파일 상태 탭을 선택합니다.

▼ 그림 3-19 깃 상태 확인



새 파일을 추가하면 여기에 등록됩니다.
즉, 워킹 디렉터리 안에 있는 파일을 의미합니다.

tracked와 unmodified 상태의 파일 표시

untracked와 modified 상태의 파일 표시

소스트리는 깃의 파일 상태를 두 영역으로 표시합니다. 스테이지의 tracked와 unmodified 상태는 스테이지에 올라간 파일에 표시합니다. untracked와 modified 상태는 스테이지에 올라가지 않은 파일에 표시합니다.

새 파일을 추가하면 untracked 상태입니다. 따라서 스테이지에 올라가지 않은 파일 목록에 출력합니다. 스테이지에 올라가지 않은 파일이란 워킹 디렉터리 안에 있는 파일을 의미합니다.

이처럼 소스트리는 직관적으로 깃에 추가된 파일을 확인할 수 있습니다. status 명령어와 관련된 구체적인 사용법은 실습을 진행해 나가면서 하나씩 설명하겠습니다.

3.5 파일 관리 목록에서 제외: .gitignore

깃은 tracked 상태인 모든 것을 추적 관리합니다. 파일뿐만 아니라 서브 폴더와 그 안의 파일들도 포함합니다. 즉, 디렉터리 전체가 모두 관리 대상입니다. 하지만 프로젝트를 하다 보면 모든 파일을 추적하지 않아야 하는 경우도 있습니다.

실제 작업하다 보면, 워킹 디렉터리에 불필요한 파일이 생성되거나 보안에 민감한 파일들이 있을 수 있습니다. 로컬 저장소를 혼자만 사용한다면 이러한 파일들은 신경 쓰지 않아도 됩니다. 하지만 자신의 저장소를 다른 사람들과 공유한다면 이 파일들은 분리해서 관리해야 합니다.

깃으로 관리하고 싶지 않은 파일과 폴더는 별도의 .gitignore 설정 파일 안에 나열해서 적어 줍니다.

3.5.1 .gitignore 파일

.gitignore는 git과 ignore(무시하다)의 합성어입니다. 워킹 디렉터리 안에 .gitignore 파일을 생성합니다. 이 파일은 앞에 점(.)이 있어 숨겨진 파일로 관리됩니다.

.gitignore 파일은 깃에서 관리하지 않는 파일들의 목록을 가지고 있습니다. 깃은 이 파일에 작성된 목록들을 추적하지 않습니다. 또 로컬 저장소를 서버로 전송하거나 다른 사람과 공유할 때도 이를 분리하여 처리합니다.

.gitignore 파일은 텍스트 에디터를 이용하여 간단하게 작성할 수 있습니다. 특별한 도구 없이 파일 이름만 .gitignore로 만들면 됩니다. 깃에서 제외할 파일 목록을 직접 적어 주거나 규칙을 사용하여 나열할 수 있습니다. .gitignore 파일을 작성할 때는 저장소 폴더의 최상위 디렉터리에 두어야 합니다.

3.5.2 .gitignore 파일 표기법

파일에서 #으로 시작하는 줄은 주석으로 처리합니다. # 없이 완전한 파일 이름을 적어 주면 그 파일은 깃의 관리 대상에서 제외됩니다. 이때 경로가 있다면 경로명도 같이 입력해야 합니다.

```
# DB 접속 파일을 제외함


dbinfo.php
```

애스터리스크(*) 기호를 사용하여 패턴을 정의할 수 있습니다. * 기호는 모든 문자열을 대체할 수 있습니다. 이러한 문자를 셸 글로빙(globbing)이라고 합니다. 글로빙 문자를 사용하여 패턴을 확장합니다.

```
# 오브젝트 파일은 제외함


*.obj
```

ignore 패턴을 작성할 때 반드시 추적 관리를 제외하는 파일만 작성하는 것은 아닙니다. 제외하지 않는 파일과 필요한 파일은 파일 이름 앞에 느낌표(!)를 사용합니다. 느낌표는 부정을 의미하는 not과 같습니다.

```
# 환경 설정 파일은 제외하면 안 됨

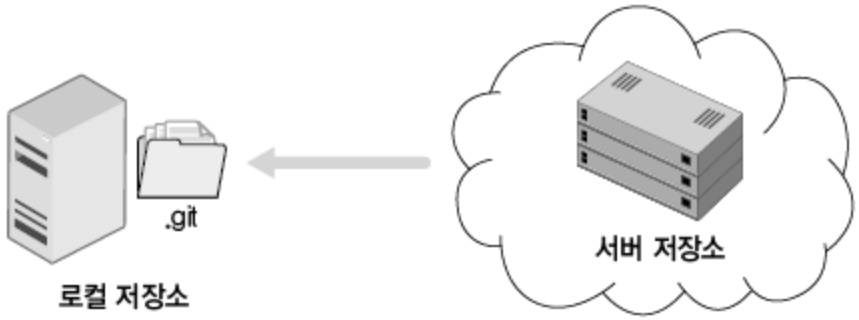

!config.php
```

3.6 깃 저장소 복제

초기화 명령(git init 명령어)은 새 저장소를 생성하는 방법 중 하나입니다.[1] 처음 프로젝트를 시작할 때는 직접 로컬 컴퓨터에 명령어를 실행하여 저장소를 생성합니다.

외부에 있는 기존 프로젝트(깃허브, 비트버킷)를 기반으로 저장소를 생성하고 싶다면 어떻게 해야 할까요? 외부 저장소를 복제해서 생성할 수 있습니다. 이처럼 외부 저장소를 이용하여 로컬 저장소를 생성하는 것을 '깃 저장소 복제'라고 합니다.

▼ 그림 3-20 외부의 깃 저장소 복제



3.6.1 공개 저장소

깃은 다수의 사람과 코드를 공유하고 협업하여 개발하는 도구입니다. 이미 깃을 기반으로 하는 공개 저장소가 여럿 있습니다. 대표적으로 깃허브, 비트버킷 같은 깃 호스팅 사이트가 있습니다(외부 저장소, 서버 관련 내용은 5장에서 설명합니다).

깃 호스팅 서비스는 공개된 저장소와 비공개된 저장소를 모두 지원합니다. 공개된 저장소는 누구나 복제하여 코드를 내려받을 수 있습니다. 요즘은 오픈 소스가 활성화되어 저장소를 공개하고 있습니다. 이미 수많은 오픈 소스를 깃으로 관리하고, 공개 저장소를 이용하여 배포합니다.

3.6.2 다운로드 vs 복제

일반적으로 공개된 소스 코드를 얻으려면 웹 사이트에서 압축 파일을 내려받습니다. 소스 코드를 내려받는다는 것은 해당 코드의 최종 복사본을 내 컴퓨터로 가져오는 것입니다. 하지만 이러한 내려받기는 깃의 이력을 포함한 저장 영역까지 내려받는 것은 아닙니다.

▼ 그림 3-21 공개된 소스 코드의 최종 복사본 내려받기



단순한 파일 내려받기로는 깃의 변경 이력까지 가져오지 않습니다.

이와 달리 깃을 이용하여 저장소를 복제하면, 최종 코드뿐만 아니라 중간에 커밋 같은 변화의 모든 이력도 같이 내려받을 수 있습니다. 또 일부 코드를 변경하여 기여하는 것도 가능할 것입니다.

3.6.3 복제 명령어

깃의 저장소를 복제하는 명령어는 clone입니다. 복제하려면 공개된 저장소의 URL이 필요합니다. 복제할 때 폴더 이름을 지정하지 않으면 공개 저장소에서 사용된 폴더와 동일한 이름으로 새 폴더를 만듭니다. 다른 이름으로 복제하길 원한다면 새 폴더 이름을 추가 인자로 적어 줍니다.

$ git clone 원격저장소URL 새폴더이름

다음은 필자가 운영하는 jinyphp 오픈 소스 사이트의 주소를 이용하여 저장소를 복제하는 예입니다. 새 폴더 이름을 적지 않아 공개 저장소에서 사용된 폴더와 동일한 이름으로 새 폴더를 만들 것입니다.

예

```
$ git clone https://github.com/jinyphp/jiny 저장소 복제



Cloning into 'jiny'...


remote: Enumerating objects: 975, done.


remote: Total 975 (delta 0), reused 0 (delta 0), pack-reused 975


Receiving objects: 100% (975/975), 4.98 MiB | 3.67 MiB/s, done.


Resolving deltas: 100% (307/307), done.
```

git clone 명령어를 사용하면 깃은 자동으로 깃 서버에 접속합니다. 그리고 저장소의 모든 소스 코드를 자동으로 내려받습니다.

깃은 저장소 안에 있는 파일들과 .git 리포지터리를 기반으로 이력을 관리합니다. 따라서 복제한 후에는 복제된 폴더 이름을 그대로 사용하지 않아도 됩니다. 필요에 따라 깃의 폴더 이름을 변경해도 괜찮습니다.

정리

이 장에서는 깃 저장소와 저장소 개념을 학습했습니다. 앞으로 깃을 학습하려면 이 장에서 배운 저장소 개념을 잘 알고 있어야 합니다. 또 각 용어들을 구분하고 이해하는 것도 필요합니다(특히 워킹 디렉터리와 스테이지 영역). 각 장의 기능을 학습하다 용어 이해가 부족하다고 느낄 때 이 장을 다시 읽어 보는 것도 좋습니다.

1 앞에서 살펴보았듯이 새 저장소는 소스트리를 이용해서 만들 수도 있습니다.